# Accelerating Large-Scale Decision Tree Induction on Heterogeneous CPU-GPU Architectures

Bruno Mirčevski[0009−0007−1008−6708] and Krzysztof Jurczuk[0000−0001−6469−1769]

Faculty of Computer Science, Bialystok University of Technology, Bialystok, Poland

**Abstract.** WIP: We investigate parallel implementations of two widely used decision tree induction algorithms, CART and C4.5, on multicore CPUs, CUDA-enabled GPUs, and heterogeneous CPU-GPU architectures.

**Keywords:** Decision trees · CART · C4.5 · Parallel computing

## 1 Introduction

Decision trees are among the most widely used models in machine learning and data mining [6]. Their predictions are easy to inspect and explain, so they suit applications in which a model must be understood or audited. They are also the base learners of random forests and gradient boosting [5,10]. CART [4] and C4.5 [18] are two of the best-known algorithms for inducing classification trees.

Growing a single tree is usually considered cheap. In practice, however, a tree is rarely trained only once. Tuning the pruning strength, depth limit or minimum leaf size needs one tree per candidate value, and $k$-fold cross-validation multiplies this by $k$. At the same time, datasets with tens of millions of samples or thousands of attributes arise in areas such as particle physics [3], bioinformatics [23] and malware detection [9]. On such data, a straightforward sequential implementation can need minutes to hours per tree, and parameter tuning pays that cost repeatedly.

Modern servers combine many cores on several sockets with high-bandwidth memory, and many are equipped with data-center GPUs. Using this hardware efficiently for tree induction is challenging, for two reasons. First, the work per node shrinks with depth. Nodes near the root hold most of the training set and provide enough work for thousands of GPU threads. The lower levels consist of many small nodes, for which kernel-launch, synchronization and transfer costs often dominate. Second, split evaluation does little computation per value read, so it is often limited by memory bandwidth.

SLIQ [14] and SPRINT [21] introduced presorting to scale to large data. Later work targeted multicore processors [2] and distributed systems [1]. Several studies proposed GPU implementations [12,16,22], but they were evaluated on GPUs of earlier generations. Most current high-performance GPU tree learners [5,10] use histogram-based split finding: continuous values are assigned to a

fixed number of bins before induction. This is cheaper, but the chosen thresholds, and therefore the trees, usually differ from those an exact CART or C4.5 search would find. To our knowledge, no public implementation grows an exact CART or C4.5 tree using the CPU and the GPU together, with each node sent to one device according to its size.

In this paper, we investigate parallel implementations of CART and C4.5 for multicore CPUs, CUDA-enabled GPUs and heterogeneous CPU–GPU systems. They search for splits exactly, without binning: every cut between two consecutive distinct attribute values is tested. All our implementations grow the same tree. We checked this node by node, against each other and against scikit-learn (CART) and Quinlan's original C4.5 program. Our speed comparisons therefore always compare runs that build the same tree.

We evaluate five implementations, from a straightforward sequential baseline to a hybrid one that uses the CPU and the GPU together. The experiments use five public datasets with up to 32 million samples and 2,381 attributes, recent CPU servers, and two generations of data-center GPUs. The optimized sequential implementation is up to 25× faster than the straightforward one. On 48 cores of a two-socket server, the multicore code is up to 40× faster again, and the GPU up to 4× faster than that. Overall, the best version is up to 1,570× faster than the straightforward sequential code. Our multicore implementation trains CART trees up to 360× faster than scikit-learn [17] and C4.5 trees up to 3,100× faster than Weka [8]. On the GPU, these factors reach 1,400× and 8,100×. For deep trees, the hybrid CPU–GPU implementation is up to 4.9× faster than growing all nodes on the GPU.

The main contributions of this paper are:

- exact parallel implementations of CART and C4.5 for multicore CPUs, GPUs and heterogeneous CPU–GPU systems, with identical trees across implementations;
- a hybrid CPU–GPU scheme that assigns the nodes of a tree to devices by size, so that the CPU and the GPU grow the tree concurrently;
- an experimental evaluation on large datasets and current hardware, covering training time and its breakdown, memory usage, scalability and energy consumption, and a comparison with established CPU and GPU implementations;
- an open-source release of the implementations.[1]

The rest of the paper is organized as follows. Section 2 reviews related work. Section 3 describes our implementations. Section 4 presents the experimental setup and Section 5 the results. Section 6 discusses the findings, and Section 7 concludes the paper.

[1] *[Repository URL.]*

## 2 Related Work

### 2.1 Decision Tree Induction: CART and C4.5

CART [4] builds a binary tree by recursive partitioning. At each node it chooses the split that most reduces Gini impurity, and it then simplifies the tree by cost-complexity pruning. C4.5 [18] chooses splits by gain ratio: the information gain divided by the entropy of the split itself. It splits a nominal attribute into one branch per value and a numeric attribute at a single threshold, and applies error-based pruning. For a numeric attribute, an exact search sorts the values in the node and evaluates every cut between consecutive distinct values. For entropy-based criteria, Fayyad and Irani [7] showed that an optimal cut lies on a boundary between classes in this order, so the remaining cuts can be skipped without changing the result. EC4.5 [19] obtained the same tree as C4.5 with a faster threshold search, a line continued by YaDT [20]. The cost of exact induction is therefore dominated by sorting and scanning the samples of every node, which the systems below address in different ways.

### 2.2 Parallel Induction on CPUs

SLIQ [14] sorts each numeric attribute once before induction and grows the tree breadth-first. It keeps the class labels in a list that must fit in memory. SPRINT [21] removes this limit by keeping one presorted list per attribute. When a node is split, each list is partitioned between the children instead of being sorted again. SPRINT also runs on clusters, with the lists divided among the machines. Both still test every distinct value as a threshold. They scale because they avoid sorting again, not because they test fewer thresholds.

Tree induction can be parallelised in two ways. Task parallelism grows different subtrees at the same time. Data parallelism splits the work within one node, by attribute or by sample. Many systems combine both. Task parallelism suffers from load imbalance, because subtrees differ in size. Data parallelism has little work to share once nodes become small. Mixed schemes therefore use data parallelism near the root and task parallelism deeper in the tree [2]. Aldinucci et al. [2] parallelised YaDT on multicore processors with the FastFlow framework. Their NP strategy processes nodes as tasks, and their NAP strategy also splits a node's attributes across threads when a cost estimate says it pays off. They call this mix "hybrid", but on a single processor. Choosing the strategy by node size anticipates our idea of choosing the device by node size.

Yggdrasil [1] runs on a Spark cluster and splits the data by attribute. Each worker keeps all values of its attributes, sorted and, where possible, compressed. It sends back only its best split. Systems that partition by sample must exchange statistics for every candidate split, so they use binning to keep that exchange small. Yggdrasil trains deep trees on thousands of attributes without binning, but on a cluster rather than within a single machine.

### 2.3 GPU-Based Tree Induction

*Single trees.* CUDT [12] ports SPRINT's attribute lists to CUDA. The CPU controls the construction and stores the tree. The GPU evaluates the splits with prefix sums and partitions the lists, one node at a time. Its speed-up is largest at the root and shrinks with depth: nodes become smaller and more numerous, and each one needs its own host–device transfers. Nasridinov et al. [16] split the work in the same way for ID3. The tree grows one node at a time: the GPU sorts the node's samples and does the calculations, and the CPU updates the tree. They call this "hybrid CPU–GPU" and report both running time and energy on a low-end GPU. Strnad and Nerat [22] instead keep the whole construction of a CART-like binary tree on the GPU, combining node-, attribute- and split-level parallelism. Nodes are processed in batches of similar size to limit load imbalance. The schedule therefore takes node size into account, but it uses only the GPU.

*Ensembles and boosting.* CudaTree [11] learns random forests on the GPU. It first grows depth-first, with the whole GPU working on one large node at a time. Below a node size chosen by a fitted model, it switches to breadth-first growth, processing a whole level at once. Its authors also ran CudaTree alongside the multicore library wiseRF, but the GPU and the CPU trained different trees of the same forest rather than sharing one tree. For gradient boosting, Mitchell and Frank [15] ported the exact greedy algorithm of XGBoost [5] to the GPU. It processes all nodes of a level together, using prefix sums over the sorted attribute values. The CPU does no computation, which avoids host–device transfers but requires the training data to fit in GPU memory. ThunderGBM [24] also names the shift from a few large nodes to many small ones as a main difficulty for GPUs. Histogram-based learners such as LightGBM [10] test only bin boundaries and are not exact.

### 2.4 Heterogeneous CPU–GPU Induction

The works above combine the CPU and the GPU in different ways. In CUDT and in Nasridinov et al., the CPU controls the construction and the GPU computes the splits. Strnad and Nerat and CudaTree organise the work by node size, but only on the GPU. CudaTree with wiseRF is the only work above in which both devices build trees at the same time, but each device builds whole trees of a forest. Presorting, level-wise GPU growth and size-aware scheduling have all been used before. Our work differs in what each device is given, namely individual nodes of one tree, and in letting the CPU grow part of the tree rather than only control it.

In our hybrid scheme, the GPU grows the large nodes of a tree level by level. Subtrees below a size threshold go to a pool of CPU threads, which grow them while the GPU continues with the large nodes. To our knowledge, no previous work divides the nodes of a single CART or C4.5 tree between the CPU and the GPU by size, with both devices working on the tree at the same time.

## 3 Our Approach

We implemented CART and C4.5 five times: Serial Simple, Serial Optimized, Multicore CPU, GPU-Based and Hybrid. All five grow exactly the same tree. They differ in how a node's samples are kept sorted, which cuts are scored, and where and in what order nodes are grown (Table 1).

All implementations share the data layout and the split rules. Attribute values are stored by attribute as 32-bit floats, and class labels are mapped to integers. One double-precision routine computes the gain of a cut on both the CPU and the GPU. Because the two can still differ in the last bit, gains within $10^{-12}$ count as equal: the earlier cut wins, then the attribute with the lower index. Every optimization below skips only cuts that provably cannot win under this rule. Parallel builders create nodes in a nondeterministic order, so nodes are renumbered in preorder at the end. A regression suite requires every implementation to reproduce the same reference trees, which match scikit-learn and Quinlan's C4.5 except where two cuts have exactly equal gain.

Pruning runs after growth, in CPU code shared by all implementations. With a fixed $\alpha$, CART also stops early at nodes with fewer than $\alpha N$ misclassified training samples, since pruning would collapse any subtree below them anyway. The pruned tree is the same, but those subtrees are never grown.

**Table 1.** The five implementations. *Presort*: each attribute is sorted once and kept sorted by stable partitioning. *Bounds*: cuts that provably cannot win are skipped. *Filter*: the remaining cuts are scored only if a cheap estimate can beat the best cut so far. Hybrid grows large nodes as GPU-Based and small subtrees as Multicore CPU.

| | Sorting | Scoring | Hardware | Parallelism | Node order |
|---|---|---|---|---|---|
| Serial Simple | each node | all cuts | 1 core | none | depth-first |
| Serial Optimized | presort | bounds, filter | 1 core | none | depth-first |
| Multicore CPU | presort | bounds, filter | cores | data, task | depth-first |
| GPU-Based | presort | bounds | GPU | data | breadth-first |
| Hybrid | presort | both | GPU + cores | data, task | both |

### 3.1 Serial Simple

Serial Simple follows the textbook algorithm and is the baseline for the others. At each node it sorts the node's samples by every attribute, scores every cut, chooses the split with the shared rule, and recurses on the two children. For a node with $n$ samples and $F$ attributes, sorting alone costs $O(Fn \log n)$, and every comparison reads an attribute value from a random memory location.

### 3.2 Serial Optimized

Serial Optimized keeps the algorithm but changes how it is executed in two ways.

*Sort once.* Before growth, every attribute is presorted with a stable radix sort. Each sorted entry holds the value together with the sample and its class, so a sweep never has to look up a label. A node occupies the same index range in every attribute. After a split, the winning attribute already lists the left samples first, and every other attribute is partitioned stably to match, which keeps both children sorted, as in SLIQ and SPRINT [14,21]. An attribute that is constant on a node is dropped for the node's entire subtree.

*Score few cuts.* The sweep over a sorted attribute skips cuts that provably cannot win. A cut between two samples of the same class with unique values is never optimal [7], and with two classes the gain is convex in the class counts, so an upper bound can rule out a whole block of consecutive cuts. For the remaining cuts, the sweep maintains a cheap gain estimate with a bounded error, and only cuts whose estimate can still beat the best cut so far are scored by the exact routine. The winning sweep also yields the class counts of both children.

Memory is allocated once: every attribute has a buffer of fixed size, and one spare buffer lets large nodes be partitioned out of place.

### 3.3 Multicore CPU

Multicore CPU runs the Serial Optimized builder on a pool of threads and combines both kinds of parallelism, in the spirit of the NAP strategy of Aldinucci et al. [2]. Nodes with at least 32,768 samples sweep and partition their attributes in parallel. When a node with at least 4,096 samples is split, one child becomes a new task and the current thread continues with the other. Data parallelism thus serves the few large nodes near the root, and task parallelism the many small subtrees below. The presort is parallel as well, and extra spare buffers let all threads partition attributes at the same time. Threads are not pinned to cores, and memory is not placed explicitly on NUMA nodes.

### 3.4 GPU-Based

GPU-Based grows the whole tree on the GPU, breadth-first: it processes all nodes of a tree level together, so that deep levels with many small nodes still keep the device busy. The attributes are presorted on the GPU as they are uploaded and stored as in Serial Optimized.

For each level, the GPU splits every node's attributes into tiles of 2,048 samples, finds the best cut of every tile, and reduces these to the best cut of every node and attribute. Only these results go to the CPU, which applies the shared CART or C4.5 rule and turns children that cannot be split into leaves. The GPU then partitions the split nodes and, in the same pass, collects the class counts for the next level. Each level therefore needs a single round trip between CPU and GPU. The sweeps skip cuts with the same bounds as Serial Optimized, applied to whole tiles, but score every remaining cut with the exact routine: a sweep in which all cuts are processed at once has no best cut so far to filter against. Data is uploaded through page-locked staging buffers, placed on the GPU's NUMA node.

### 3.5 Hybrid

Hybrid uses the same GPU builder but hands each child with fewer than $m$ samples to the Multicore CPU builder, while the GPU continues with the large nodes of the next level. A GPU level costs about the same for a thousand mid-size nodes as for a few large ones, whereas CPU threads grow small subtrees independently. By default, $m$ is $N/8192$ clamped to $[1024, 2048]$ for CART and $N/4096$ clamped to $[1024, 4096]$ for C4.5; GPU-Based is the special case $m = 1$.

Hand-offs are asynchronous. For each level, the GPU gathers the attributes that can still split the handed-off children and copies them to host memory in the background; once the copy is complete, CPU tasks start growing these subtrees. The GPU does not wait for them. Nodes from both devices are renumbered in preorder once both finish.

## 4 Experiment Design

### 4.1 Datasets

We evaluate the implementations on five real-world datasets and one synthetic dataset, summarized in Table 2. All attributes are numeric.

SUSY and HIGGS [3] contain simulated particle collision events labelled as signal or background. Following the UCI repository, the last 500,000 samples of each serve as the test set. ECBDL'14 [23] originates from the protein structure prediction task of the ECBDL'14 Big Data Competition; its nominal attributes are encoded as integers. MNIST8M [13] consists of 8.1 million handwritten digits generated by deforming the MNIST training images, and SOREL-20M [9] is a malware detection benchmark built from Windows executables. These three datasets are used with their official test sets.

The training sets of ECBDL'14 and SOREL-20M do not fit into the 80 GB memory of our GPUs. To include them in the GPU experiments, we train on their first 15 million and 4 million samples, respectively, and evaluate the trees on the full test sets.

Chess is a synthetic $3 \times 3$ checkerboard: two attributes drawn uniformly from $[0, 3]$ and two classes alternating between the unit squares, without noise. Its exact tree has only nine leaves, so the training time reflects the cost per sample rather than tree growth. We use sizes from $10^6$ to $2 \cdot 10^9$ samples, for timing only.

### 4.2 Hardware

*[Four nodes of the mordor3 cluster (Table 3), each used whole (exclusive jobs). GPU nodes run the CUDA backend only, CPU nodes the CPU backends and the other CPU implementations. GPU runs are bound to the GPU's NUMA node (16 CPUs on hopper, 24 on ampere). Add measured memory bandwidth (STREAM) per node. Software: Ubuntu 24.04, Linux 7.0, g++ 13.3 (`-O3`), CUDA 12.9, NVIDIA driver 615.71.]*

**Table 2.** Datasets. Size: the training set as float32.

| Dataset | Attributes | Classes | Training samples | Test samples | Size |
|---|---|---|---|---|---|
| SUSY [3] | 18 | 2 | 4,500,000 | 500,000 | 0.3 GB |
| HIGGS [3] | 28 | 2 | 10,500,000 | 500,000 | 1.2 GB |
| ECBDL'14 [23] | 631 | 2 | 31,992,921 | 2,897,917 | 80.8 GB |
| GPU subsample | 631 | 2 | 15,000,000 | 2,897,917 | 37.9 GB |
| MNIST8M [13] | 784 | 10 | 8,100,000 | 10,000 | 25.4 GB |
| SOREL-20M [9] | 2381 | 2 | 12,699,013 | 4,195,042 | 120.9 GB |
| GPU subsample | 2381 | 2 | 4,000,000 | 4,195,042 | 38.1 GB |
| CHESS 3×3 | 2 | 2 | $10^6$–$2 \cdot 10^9$ | – | 8 MB–16 GB |

**Table 3.** Hardware. Every node has two sockets with SMT disabled; CPU, cores and L3 are per socket. Clock is base/max turbo in GHz. One GPU is used per run.

| Node | CPU | Cores | Clock | L3 | RAM | GPU |
|---|---|---|---|---|---|---|
| hopper | Xeon Gold 6444Y | 16 | 3.6/4.0 | 45 MB | 1 TB DDR5-4800 | H100 80 GB |
| ampere | EPYC 7402 | 24 | 2.8/3.35 | 128 MB | 1 TB DDR4-3200 | A100 80 GB |
| epyc | EPYC 9255 | 24 | 3.25/4.3 | 128 MB | 512 GB DDR5-6400 | – |
| dell | Xeon Gold 6542Y | 24 | 2.9/4.1 | 60 MB | 1 TB DDR5-5200 | – |

### 4.3 Compared Implementations

*[./tree backends (`-simple`, `-serial`, `-parallel`, `-cuda`, `-gpu-only`); scikit-learn and Weka J48 on the CPU; cuML and XGBoost on the GPU, with their settings. Why each is the closest equivalent (no public GPU C4.5; J48 depth limit adapter; XGBoost squared error = Gini on bins). rpart and YaDT: add or say why not.]*

### 4.4 Protocol and Measurement

*[One tree, depth ≤ 10, no pruning, all samples and attributes; what is timed (training only, from data in memory; GPU setup and upload included); peak RAM and VRAM; runs per case, means and spread; exclusive nodes; fairness (CPU threads of the hybrid, warm library processes, input layout). Source: docs/GPU-LIBRARIES.md, bench/README.md.]*

## 5 Results

### 5.1 Training Time

*[Overview of every implementation on every dataset (Tables 4 and 5): speed-ups of Serial Optimized over Serial Simple, Multicore CPU over Serial Optimized, GPU over Multicore CPU, and of this work over the existing implementations. The tables give means only (at least 10 runs per cell).]*

**Table 4.** Training time of one CART tree (depth $\leq 10$, no pruning), mean in seconds. † marks a rough estimate (no run yet); – not possible (data larger than the GPU's 80 GB, or no such algorithm). CPU runs on epyc1 (Multicore CPU 48 threads, the rest one thread), GPU runs on the H100 (Hybrid with 16 CPU threads). Placeholder: every cell will have at least 10 runs.

| **This work** | | | | | |
|---|---|---|---|---|---|
| Dataset | Serial Simple | Serial Optimized | Multicore CPU | GPU-Based | Hybrid |
| SUSY | 70.8 | 2.84 | 0.239 | 0.082 | 0.084 |
| HIGGS | 236 | 10.4 | 0.607 | 0.150 | 0.151 |
| ECBDL'14 | 5 860 | 503 | 13.5 | – | – |
| 15M subsample | 2 270 | 225 | 6.32 | 2.97 | 2.97 |
| MNIST8M | 1 880 | 234 | 5.92 | 3.57 | 3.56 |
| SOREL-20M | 12 500 | 878 | 21.8 | – | – |
| 4M subsample | 2 920 | 234 | 6.43 | 2.87 | 2.87 |
| **Existing implementations** | | | | | |
| Dataset | scikit-learn | rpart | | cuML | XGBoost |
| SUSY | 86.5 | *140*† | | 0.133 | 1.43 |
| HIGGS | 215 | *350*† | | 0.456 | 3.43 |
| ECBDL'14 | *4 500*† | *7 500*† | | – | – |
| 15M subsample | 2 200 | *3 600*† | | 9.95 | 42.2 |
| MNIST8M | *2 500*† | *4 000*† | | *7*† | *50*† |
| SOREL-20M | *13 000*† | *21 000*† | | – | – |
| 4M subsample | *3 500*† | *6 000*† | | *10*† | *45*† |

**Table 5.** Training time of one C4.5 tree (depth $\leq 10$, no pruning), mean in seconds. † marks a rough estimate (no run yet); – not possible (data larger than the GPU's 80 GB, or no such algorithm). CPU runs on epyc1 (Multicore CPU 48 threads, the rest one thread), GPU runs on the H100 (Hybrid with 16 CPU threads). Placeholder: every cell will have at least 10 runs. cuML grows an entropy tree, its closest equivalent.

| **This work** | | | | | |
|---|---|---|---|---|---|
| Dataset | Serial Simple | Serial Optimized | Multicore CPU | GPU-Based | Hybrid |
| SUSY | 77.2 | 3.10 | 0.249 | 0.109 | 0.112 |
| HIGGS | 257 | 11.9 | 0.590 | 0.164 | 0.170 |
| ECBDL'14 | 6 890 | 623 | 17.6 | – | – |
| 15M subsample | 2 590 | 277 | 7.81 | 2.99 | 2.99 |
| MNIST8M | 1 970 | 209 | 5.64 | 3.71 | 3.68 |
| SOREL-20M | 12 900 | 1 130 | 28.8 | – | – |
| 4M subsample | 3 090 | 288 | 7.85 | 3.88 | 4.30 |
| **Existing implementations** | | | | | |
| Dataset | Weka J48 | YaDT | | cuML | |
| SUSY | 230 | *46*† | | 0.128 | |
| HIGGS | 830 | *170*† | | 0.433 | |
| ECBDL'14 | *50 000*† | *10 000*† | | – | |
| 15M subsample | 24 200 | *4 800*† | | 9.89 | |
| MNIST8M | *14 000*† | *2 700*† | | *7*† | |
| SOREL-20M | *60 000*† | *12 000*† | | – | |
| 4M subsample | *18 000*† | *3 800*† | | *10*† | |

### 5.2 Comparison with Other CPU Implementations

*[./tree's backends against scikit-learn (CART) and Weka J48 (C4.5): time, memory, test accuracy, same trees.]*

### 5.3 Comparison with GPU Libraries

*[Against cuML and XGBoost: one tree, and per tree of a 10-tree forest; accuracy of exact versus binned trees; VRAM.]*

### 5.4 Performance Across Hardware

*[All ./tree backends on the four nodes, per dataset: serial, one and two sockets, H100 and A100 hybrid and GPU-based. CPU against GPU; which hardware wins for which data shape (samples, attributes, classes).]*

### 5.5 Scalability

*[Thread scaling 1–48 on epyc1 and dell1 (memory bandwidth bound); growth with the number of samples (subsamples of one dataset).]*

### 5.6 Profiling

*[Where the time goes: presort, split search, partition, CPU–GPU transfer; GPU share of the hybrid; achieved bandwidth against peak.]*

### 5.7 Energy Consumption

*[Whole-server power while each implementation trains one tree on ECBDL'14 15M (the one dataset every implementation runs on, GPUs included). Method: power source and sampling rate (BMC/IPMI for the server, RAPL for the CPUs, `nvidia-smi` for the GPU), idle power measured before the runs and subtracted, energy = integral of power over the training time, 10 runs each. The CPU implementations run on a CPU node, the GPU ones on a GPU node, so the totals compare servers, not only processors. Discuss energy per tree: a fast run at high power can still use the least energy.]*

## 6 Discussion

## 7 Conclusion

## References

1. Abuzaid, F., Bradley, J.K., Liang, F.T., Feng, A., Yang, L., Zaharia, M., Talwalkar, A.S.: Yggdrasil: An optimized system for training deep decision trees at scale. In: Advances in Neural Information Processing Systems 29 (NIPS 2016). pp. 3817–3825 (2016)

**Table 6.** Energy of one tree on ECBDL'14 15M (depth $\leq$ 10, no pruning): mean training time $t$, mean power $P$ of the whole server above idle, and energy $E = P \cdot t$. Placeholder: no measurements yet.

| | | CART | | | C4.5 | | |
|---|---|---|---|---|---|---|---|
| Implementation | Node | $t$ (s) | $P$ (W) | $E$ (kJ) | $t$ (s) | $P$ (W) | $E$ (kJ) |
| Serial Simple | epyc1 | 2 332 | 140 | 327 | 2 639 | 141 | 372 |
| Serial Optimized | epyc1 | 225 | 147 | 33.2 | 278 | 147 | 40.8 |
| Multicore CPU | epyc1 | 6.44 | 479 | 3.08 | 7.91 | 485 | 3.83 |
| GPU-Based | hopper | 2.99 | 316 | 0.95 | 3.03 | 317 | 0.96 |
| Hybrid | hopper | 3.01 | 387 | 1.16 | 3.02 | 394 | 1.19 |


2. Aldinucci, M., Ruggieri, S., Torquati, M.: Decision tree building on multi-core using FastFlow. Concurrency and Computation: Practice and Experience **26**(3), 800–820 (2014). https://doi.org/10.1002/cpe.3063
3. Baldi, P., Sadowski, P., Whiteson, D.: Searching for exotic particles in high-energy physics with deep learning. Nature Communications **5**, 4308 (2014)
4. Breiman, L., Friedman, J.H., Olshen, R.A., Stone, C.J.: Classification and Regression Trees. Wadsworth, Belmont, CA (1984)
5. Chen, T., Guestrin, C.: XGBoost: A scalable tree boosting system. In: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. pp. 785–794 (2016)
6. Costa, V.G., Pedreira, C.E.: Recent advances in decision trees: an updated survey. Artificial Intelligence Review **56**(5), 4765–4800 (2023). https://doi.org/10.1007/s10462-022-10275-5
7. Fayyad, U.M., Irani, K.B.: On the handling of continuous-valued attributes in decision tree generation. Machine Learning **8**, 87–102 (1992)
8. Hall, M., Frank, E., Holmes, G., Pfahringer, B., Reutemann, P., Witten, I.H.: The WEKA data mining software: An update. SIGKDD Explorations **11**(1), 10–18 (2009)
9. Harang, R., Rudd, E.M.: SOREL-20M: A large scale benchmark dataset for malicious PE detection (2020), arXiv:2012.07634
10. Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., Liu, T.Y.: LightGBM: A highly efficient gradient boosting decision tree. In: Advances in Neural Information Processing Systems 30. pp. 3146–3154 (2017)
11. Liao, Y., Rubinsteyn, A., Power, R., Li, J.: Learning random forests on the GPU. In: Big Learning Workshop, NIPS (2013)
12. Lo, W.T., Chang, Y.S., Sheu, R.K., Chiu, C.C., Yuan, S.M.: CUDT: A CUDA based decision tree algorithm. The Scientific World Journal **2014**, 745640 (2014). https://doi.org/10.1155/2014/745640
13. Loosli, G., Canu, S., Bottou, L.: Training invariant support vector machines using selective sampling. In: Large Scale Kernel Machines, pp. 301–320. MIT Press (2007)
14. Mehta, M., Agrawal, R., Rissanen, J.: SLIQ: A fast scalable classifier for data mining. In: Advances in Database Technology – EDBT'96. LNCS, vol. 1057, pp. 18–32. Springer (1996)
15. Mitchell, R., Frank, E.: Accelerating the XGBoost algorithm using GPU computing. PeerJ Computer Science **3**, e127 (2017). https://doi.org/10.7717/peerj-cs.127

16. Nasridinov, A., Lee, Y., Park, Y.H.: Decision tree construction on GPU: ubiquitous parallel computing approach. Computing **96**(5), 403–413 (2014). https://doi.org/10.1007/s00607-013-0343-z
17. Pedregosa, F., et al.: Scikit-learn: Machine learning in Python. Journal of Machine Learning Research **12**, 2825–2830 (2011)
18. Quinlan, J.R.: C4.5: Programs for Machine Learning. Morgan Kaufmann, San Mateo, CA (1993)
19. Ruggieri, S.: Efficient C4.5. IEEE Transactions on Knowledge and Data Engineering **14**(2), 438–444 (2002)
20. Ruggieri, S.: YaDT: Yet another decision tree builder. In: Proceedings of the 16th IEEE International Conference on Tools with Artificial Intelligence (ICTAI). pp. 260–265 (2004)
21. Shafer, J., Agrawal, R., Mehta, M.: SPRINT: A scalable parallel classifier for data mining. In: Proceedings of the 22nd International Conference on Very Large Data Bases (VLDB). pp. 544–555 (1996)
22. Strnad, D., Nerat, A.: Parallel construction of classification trees on a GPU. Concurrency and Computation: Practice and Experience **28**(5), 1417–1436 (2016). https://doi.org/10.1002/cpe.3660
23. Triguero, I., del Río, S., López, V., Bacardit, J., Benítez, J.M., Herrera, F.: ROSEFW-RF: The winner algorithm for the ECBDL'14 big data competition. Knowledge-Based Systems **87**, 69–79 (2015)
24. Wen, Z., Shi, J., He, B., Chen, J., Ramamohanarao, K., Li, Q.: Exploiting GPUs for efficient gradient boosting decision tree training. IEEE Transactions on Parallel and Distributed Systems **30**(12), 2706–2717 (2019). https://doi.org/10.1109/TPDS.2019.2920131

# 8 TEMPORARY: Chart Style Tests

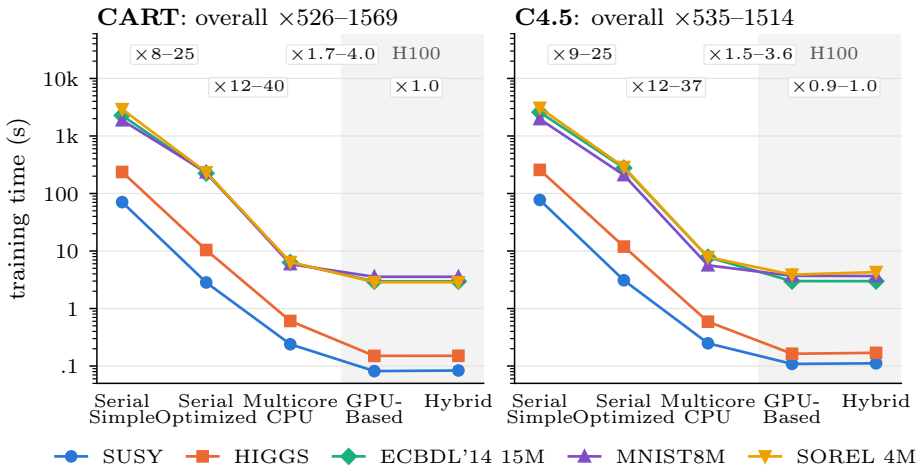



**Fig. 1.** Training time of one tree (depth $\leq 10$, unpruned) through the steps of this work: Serial Simple, Serial Optimized and Multicore CPU on epyc1, then GPU-Based and Hybrid on the H100 (grey). Boxes: range of the step's speed-up over the five datasets.

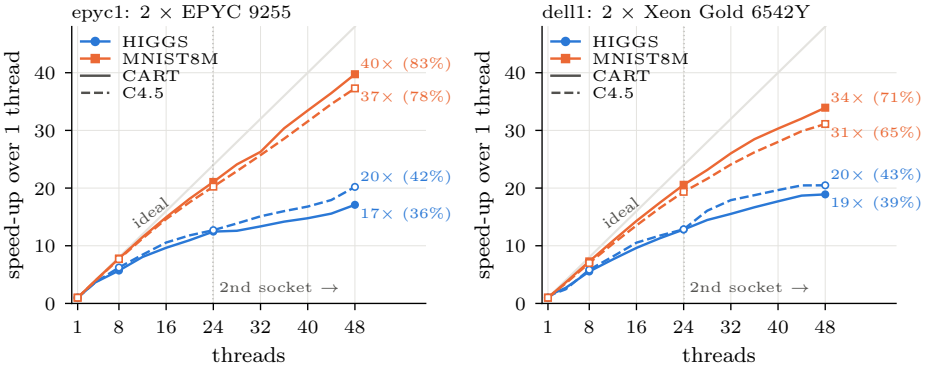



**Fig. 2.** Speed-up of Multicore CPU over one thread on epyc1 (left) and dell1 (right), depth $\leq 10$, unpruned. Labels: speed-up and parallel efficiency at 48 threads; dotted line: the second socket starts at 25 threads.

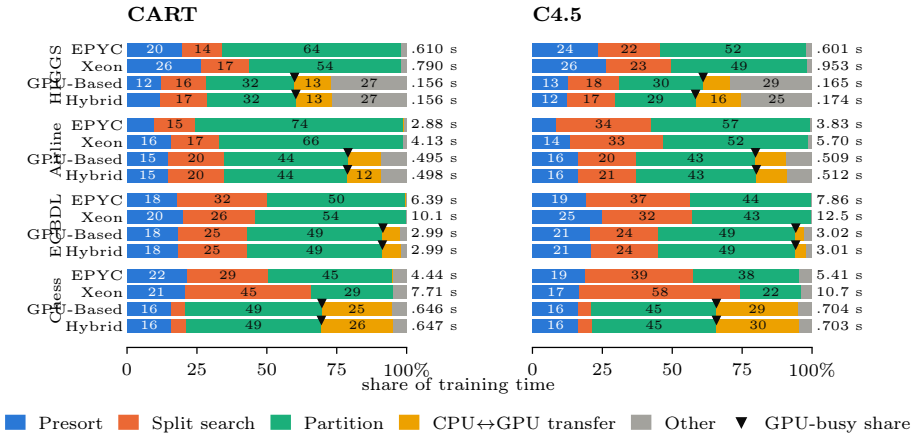



**Fig. 3.** Where the training time goes (percent of the training time; mean of the profiled runs, depth $\leq 10$, unpruned). EPYC, Xeon: Multicore CPU with 48 threads on epyc1 and dell1; GPU-Based and Hybrid on the H100. Triangle: share of the time the GPU is busy. Chess: 3×3 board, $10^9$ samples.

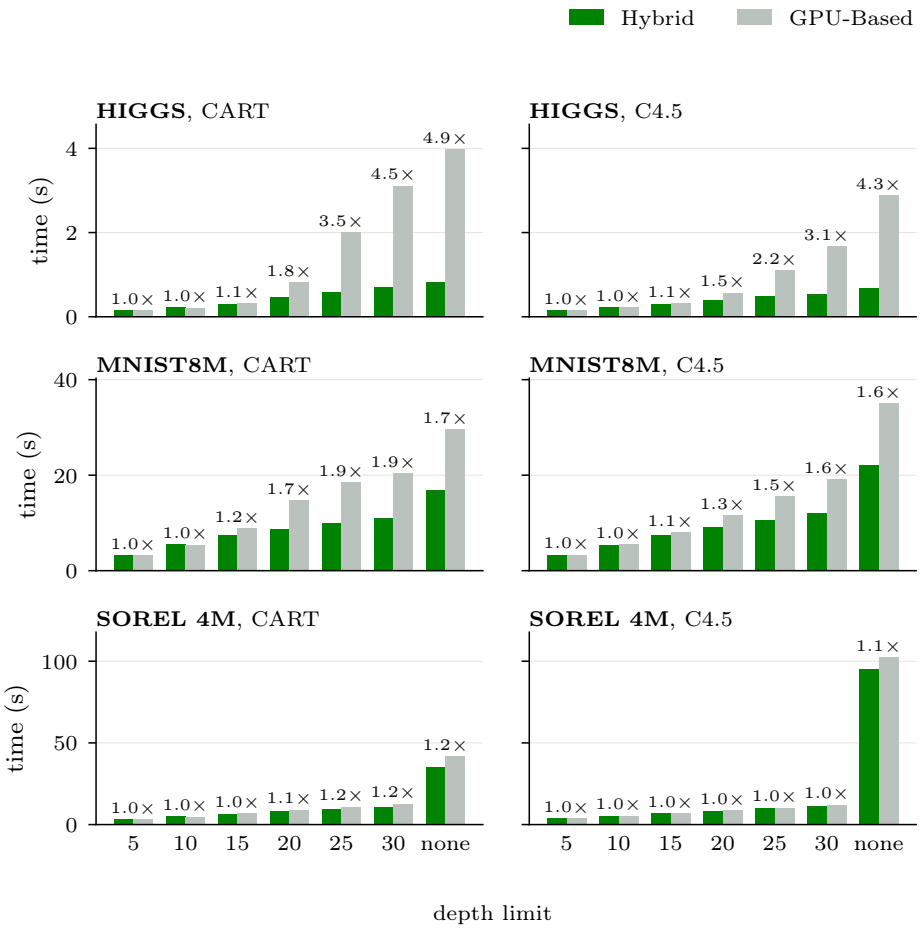



**Fig. 4.** Hybrid against GPU-Based by depth limit on the A100 for HIGGS, MNIST8M and the SOREL-20M 4M subsample (unpruned, mean of 3 runs, all 48 CPU threads of the node; MNIST8M at depth 5 and 10 with the 24 of the GPU's socket). Numbers above the bars: how many times slower GPU-Based is. Both modes grow the same trees.

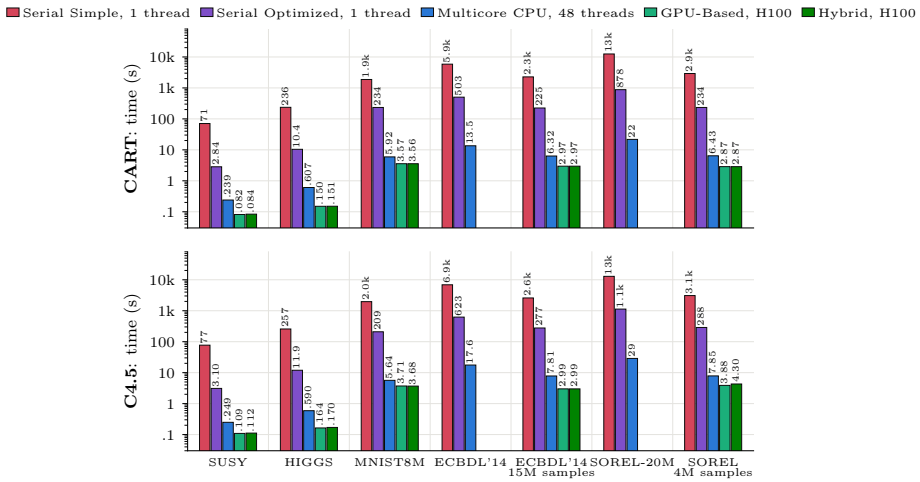



**Fig. 5.** Training time of one tree (depth $\leq 10$, unpruned), log scale: mean over the runs. Serial Simple, Serial Optimized and Multicore CPU on epyc1, GPU-Based and Hybrid on the H100 (Hybrid with 16 CPU threads). No bar: no run (the full ECBDL'14 and SOREL-20M do not fit in the 80 GB of VRAM).

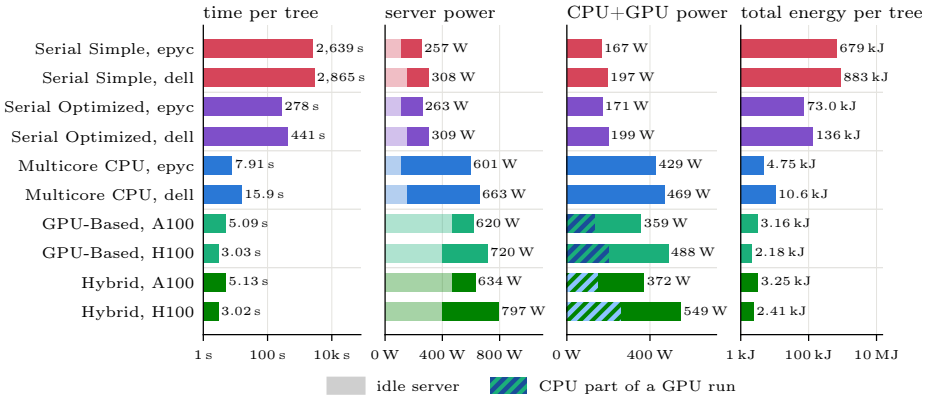



**Fig. 6.** Training one C4.5 tree on ECBDL'14 15M (depth $\leq 10$, unpruned). Server power and energy: the whole server at the PSU, idle included. CPU+GPU power: CPU packages (RAPL) and the GPU in use (NVML). CPU implementations on epyc and dell, GPU implementations on ampere (A100) and hopper (H100).